dgf-factory — Product Description

What the product is, what it installs and may write, how its gate decides, and where it stops — as built at version 0.2.0

dotGov Solutions — dgf-factory
3 November 2026

---

1. What it is
2. How it is delivered
3. The install model
4. The skills
5. The validators and the gate
6. The learning loop
7. Rules, estate documentation, QA and security
8. Prerequisites
9. Documented limits
10. Security posture
Appendix A — Scripts and exit codes
Appendix B — Glossary

This paper describes `@andrian-org/dgf-factory` 0.2.0 as the code stands on 3 November 2026. Decisions and limits are cited to the project's architecture decision records, as "(ADR 0029)". Anything not yet built is named as such.

1. What it is

`dgf-factory` is an npm command-line tool that installs an AI-assisted delivery pipeline for the **DotGov Framework (DGF)** into a project. DGF applications are composed largely from configuration held in *workspaces*: modern JSON component configuration under `FM/_COMPONENTS/`, and legacy XML for forms, workflows, processes, entity settings and views. A *workspaces root* holds the base workspace, `webasm`, and one workspace per application; a change to `webasm` is live in every application.

The tool puts three things into a project:

- **22 skills** — prompt-programs (`SKILL.md` files) that an AI coding harness such as Claude Code, Codex or Gemini CLI runs as commands (§4) — and, for Claude Code, **two agents** that run independent plan tasks in parallel.
- **A runtime** in `.dgf-factory/runtime/`: Python scripts that validate DGF configuration in both schema families, check plans and changes and compute the gate, and a **knowledge base** of DGF facts, each file stamped with the DGF version it was read from, with the vendored JSON Schemas and XSDs.
- **A receipt**, `.dgf-factory/install.json`, recording every file written, so an update never overwrites a file the user edited.

Determinism before prompting. Whatever a JSON Schema, an XSD or a script can decide is decided by a script: the plan format, scope, whether a new file is JSON or XML, validation, which findings a branch introduced, and the gate's status. The model writes configuration, makes the judgements no script can, and relays the scripts' verdicts verbatim. The gate's machine-readable block is built by one Python library, never by a prompt (ADR 0022).

The unit of work is the whole workspaces root, because one file can break another it never touched (ADR 0026). The gate validates the whole root and blocks only on what the branch introduced, compared with the merge-base (ADR 0018).

Each stage of the pipeline writes an artifact on disk that the next stage reads:

```
/dgf  ──►  /dgf-plan  ──►  /dgf-implement  ──►  /dgf-verify  ──►  /dgf-commit
 set up     plan the       compose, task by     the gate: plan,    conventional
 the root   change         task, and validate   change, baseline   commits
                                                  │     ▲
                                                  ▼     │
                                                /dgf-fix ──► patches/ ──► /dgf-evolve ──► skill-
context/
                                                fix inside              distil into rules that
                                                the plan                only tighten the skills
```

The architecture is derived from the open-source AI Factory pipeline — a derivation, not a dependency: nothing at run time requires AI Factory or any other DGF agent-tooling effort (ADR 0029 §9).

2. How it is delivered

2.1 The package

`@andrian-org/dgf-factory` 0.2.0, binary `dgf-factory`, is internal (`UNLICENSED`) and distributed through GitHub Packages (ADR 0029 §1); a repository workflow publishes it when a tag `dgf-factory-v<version>` is pushed, after the test suites and the package check pass. The CLI is plain Node.js ES modules for Node ≥ 18.3, with no runtime dependencies and no build step. The payload is `bin/`, `cli/`, `skills/`, `scripts/`, `knowledge/` and `agents/` (ADR 0030); tests, provenance ledgers, maintainer tools and documentation are not shipped.

Installing needs a GitHub token with `read:packages`, once per machine:

```
npm config set @andrian-org:registry https://npm.pkg.github.com
npm login --scope=@andrian-org --auth-type=legacy --registry=https://npm.pkg.github.com
npm install -g @andrian-org/dgf-factory
```

No shipped file names a path into the DGF repository; shipped content cites the knowledge base or a DGF docs MCP tool instead, and the doctor blocks on a violation (ADR 0030).

2.2 The commands

Command	Does	Options
`setup`	Install or update: preview, write, build the environment, receipt last	`--harness` `--root` `--yes` `--dry-run` `--no-python`
`status`	Compare every installed file and MCP entry with the receipt; writes nothing	`--root` `--json`
`uninstall`	Remove only what setup wrote and nobody edited	`--harness` `--root` `--yes` `--dry-run`

All take `--help`; the CLI takes `--version`. Exit codes are `0` clean, `1` blocked, `2` warnings, `3` usage error; `DEBUG=1` traces to standard error.

- **`setup`** asks for harnesses on a terminal; otherwise `--harness` and `--yes` are required. `--yes` applies creates, updates, recreations and unedited removals — **never a conflict**. `--no-python` prints the environment's command instead of running it. Setup refuses the filesystem root and the home directory, exits `2` when it kept an edited file, left a manual MCP step or could not build the environment, and `1` when an MCP file does not parse or a write fails. It never runs the `/dgf` interview (ADR 0029 §1).
- **`status`** also compares the receipt's version with the CLI's; it exits `0` when all matches, `2` on any difference or version drift, `3` with no receipt.
- **`uninstall`** removes the `dgf-mcp` entry only while it is unchanged and no other installed harness uses that file, and the runtime only when no harness is left. It never touches `config.yaml`, `DESCRIPTION.md`, `plans/`, `patches/`, `skill-context/` or `evolutions/`, and lists the edited files and merged MCP files it left behind.

2.3 The harnesses

Setup knows 16 harnesses, following AI Factory 2.21.0's registry folder for folder (ADR 0029 §3 and errata). It writes each skill into the harness's skills folder and an HTTP `dgf-mcp` entry into the harness's project-level MCP file, in that harness's documented shape; where no file can be written safely, it prints a manual step.

Harness	`--harness`	Skills folder	`dgf-mcp` entry
Claude Code	`claude`	`.claude/skills/` (agents: `.claude/agents/`)	`.mcp.json`
Cursor	`cursor`	`.cursor/skills/`	`.cursor/mcp.json`
Codex CLI	`codex`	`.codex/skills/`	`.codex/config.toml` [1]
Codex app	`codex-app`	`.agents/skills/`	`.codex/config.toml` [1]
GitHub Copilot	`copilot`	`.github/skills/`	`.vscode/mcp.json`
Gemini CLI	`gemini`	`.gemini/skills/`	`.gemini/settings.json`
Junie	`junie`	`.junie/skills/`	`.junie/mcp/mcp.json`

Harness	`--harness`	Skills folder	`dgf-mcp` entry
Qwen Code	`qwen`	`.qwen/skills/`	`.qwen/settings.json`
Devin Desktop (formerly Windsurf)	`windsurf`	`.devin/skills/`	manual — user-level config only
Warp	`warp`	`.warp/skills/`	`.warp/.mcp.json`
Zencoder	`zencoder`	`.zencoder/skills/`	manual — docs not verifiable
Roo Code [2]	`roocode`	`.roo/skills/`	`.roo/mcp.json`
Kilo Code	`kilocode`	`.kilocode/skills/`	manual — JSON with comments
Antigravity 2.0	`antigravity`	`.agents/skills/`	`.agents/mcp_config.json`
OpenCode	`opencode`	`.opencode/skills/`	`opencode.json`
Universal / Other	`universal`	`.agents/skills/`	`.mcp.json`

[1] Codex reads a project's `.codex/config.toml` only once the user trusts the project. [2] ADR 0029's errata record that Roo Code itself was shut down on 2026-05-15; the entry remains.

Skills name commands as `/dgf-…`; setup rewrites that form in each body to `$dgf-…` for `codex` and `codex-app` and to `/skills dgf-…` for `qwen`. `codex-app` cannot be chosen with `universal` or `antigravity`: they share `.agents/skills/` but render skills differently.

2.4 What is committed

Written	Where	Committed
A rendered `SKILL.md` per skill, per harness; the Claude Code agents	each harness's folder	yes
Validators, knowledge base, skill references and templates	`.dgf-factory/runtime/`	yes
The validators' Python environment	`.dgf-factory/runtime/.venv/`	no — a managed `runtime/.gitignore`
The receipt	`.dgf-factory/install.json`	yes
A `dgf-mcp` entry	the harness's MCP file	the project's choice
The pipeline's artifacts: config, plans, patches, overrides, rules, docs	`.dgf-factory/`	yes
Security scan reports	`.dgf-factory/security/`	no — a `.gitignore` of `*`

A teammate who clones a set-up project needs no CLI, only Python to build `.venv`. The cost (ADR 0029): about 4.4 MB committed per project, and every CLI update is a change to commit. Setup never edits the project's `.gitignore`; it prints the step of adding `.claude/worktrees/` for parallel execution.

3. The install model

The runtime mirrors the payload exactly — `scripts/`, `knowledge/` and every skill's folder, untransformed `SKILL.md` included, because `/dgf-evolve` reads a skill's shipped text there and three scripts find their siblings by position (ADR 0029 §2).

Rendering. Skills write `{{dgf_runtime}}` wherever they name a runtime path. Setup replaces it in every installed Markdown file with the project-relative `.dgf-factory/runtime` — relative, because a skill runs from the project root and an absolute path would break once committed. Per harness, only the body's command form is rewritten; frontmatter is kept (ADR 0029 §3).

The receipt records, per file, its source, its owner (`runtime` or a harness) and **two hashes**: `sourceHash` of the package file and `installedHash` of the file as written; and per harness, the MCP file, key and entry hash. Hashes are SHA-256 over text normalised to LF without trailing whitespace at the end, or over raw bytes for binary files. The Node CLI and the Python doctor implement the hash independently, held to one shared fixture. Keys must be project-relative; an absolute path or a `..` segment is refused.

Updating is re-running `setup`, each file decided by ADR 0029 §5's table:

Package file changed	User edited it	Action
no	no	leave it
yes	no	rewrite it; record both new hashes
no	yes	keep the user's file; report it as edited
yes	yes	keep the user's file; report a conflict with the new template's diff
file missing	—	report it; recreate only if the user agrees
removed from the package	—	report it; delete only if unedited and the user agrees

A file at a target path the receipt does not list, and that differs from what would be written, is an unmanaged conflict and is never overwritten.

Writing. Setup previews every create, change and removal, conflicts first, and writes only after the user agrees. Each file is written to a temporary file in the same folder and renamed into place; a target that a symlink takes outside the project is refused. The receipt is written last, so it lists only complete files, and a failed run resumes from it. Files the tool does not own are parsed, never replaced: a JSON settings file that does not parse is left byte-for-byte, TOML is only appended to, and a differing `dgf-mcp` entry is reported as a conflict (ADR 0029 §6–7).

The project virtual environment (ADR 0031). The validators need Python 3.9+ with `lxml` and `jsonschema`, pinned with hashes. Setup finds a 3.9+ interpreter, creates `.dgf-factory/runtime/.venv`, and installs with `pip --require-hashes` — skipped when both packages already import, so a re-run needs no network. pip never runs outside that environment. Every entry

script switches itself to the environment's interpreter before importing an engine, so skills call a plain `python3`. A validator without its dependencies exits `3` with the install command: reported, never passed.

The install footprint — everything under `.dgf-factory/` and every path the receipt lists — is not the estate's configuration: the change checks ignore it, and `/dgf-scaffold` counts a folder holding only it as empty (ADR 0029 §2).

The doctor. `/dgf-doctor` runs the standard-library-only `doctor.py` over five required sections — install (every file the receipt lists, every skill for every harness), skill slices, portability, line endings, the validator runtime — and ends with a `doctor` gate block whose suggested next command is always `null`: a broken install is fixed by re-running setup (ADR 0022 §8).

4. The skills

A skill's `allowed-tools` lists what it may run **without a prompt**. It pre-approves and does not restrict: a command no rule matches still runs after the user approves it. Scripts are pre-approved only as `python3 ".dgf-factory/runtime/scripts/*"`, never `python3 *`. "May write" below is taken from each skill's *Artifact Ownership* section. `config.yaml`, `DESCRIPTION.md`, `RULES.md`, `RESEARCH.md`, `qa/`, `docs/`, `skill-context/` and `evolutions/` are under `.dgf-factory/`. Only `verify` and `doctor` are gates.

4.1 The pipeline spine and the install check

Skill	Purpose	May write
`/dgf`	Inventory the root; record its DGF version	`config.yaml`, `DESCRIPTION.md`
`/dgf-plan`	Plan a change — fast, full or ultra — on its own branch	the plan; never a workspace file
`/dgf-implement`	Execute the plan task by task, validating each	the task's listed files and deletions; checkboxes
`/dgf-verify`	The change gate over the whole root	nothing; relays the `verify` gate block
`/dgf-commit`	Commits per the Commit Plan; offers a push	git history, after the user confirms
`/dgf-doctor`	Check the install against the receipt	nothing; relays the `doctor` gate block

4.2 The learning loop

Skill	Purpose	May write
`/dgf-fix`	Reproduce, fix in the plan's scope, confirm; or `--record`	files its scope check passed; one patch
`/dgf-evolve`	Distil patches into overrides; only when invoked	`skill-context/`, `evolutions/`

4.3 DGF-specific skills

Skill	Purpose	May write
`/dgf-component`	Inspect, validate or scaffold a component	one new component file, in scope
`/dgf-process`	Inspect, validate, author or modify a process or workflow	`process.xml`, `_workflow.xml`, in scope
`/dgf-model`	Inspect, validate, add or modify an entity	an entity's `settings.xml`, in scope
`/dgf-audit`	The root's health, or a file's blast radius	nothing; a report
`/dgf-scaffold`	A new application in an empty folder	the design file and the generated app

`/dgf-component` works in both families and is parity-aware; `/dgf-process` shows a workflow's reach before modifying it; `/dgf-model` writes no SQL; `/dgf-scaffold` writes `docs/application.json` and, through its generator only, the application, and never `.dgf-factory/`. Inspect and validate run anywhere. Writing modes run only inside the active plan, with `check_change.py` deciding the scope **before** the write; none ticks a checkbox, deletes a file, converts a file's family, or emits a gate block. Without git, `/dgf-process` and `/dgf-model` settle a write against a baseline saved just before it, `.dgf-factory/baseline.json` (ADR 0025). Each artifact kind has one writer, which stops on anything else (ADR 0027 §2): processes and workflows are `/dgf-process`'s; entity settings `/dgf-model`'s; JSON components, forms, views, grid forms and options `/dgf-component`'s. `/dgf-implement` and `/dgf-fix` may write any of them for a plan task or a fix.

4.4 Quality and authoring skills

None is a gate, none pre-approves git — the diff comes from `show_change.py` — and none reads an override (ADR 0032, ADR 0034).

Skill	Purpose	May write
`/dgf-review`	A DGF reviewer's read of a change	nothing; a report
`/dgf-rules`	Seed, add, edit or demote rules; only when invoked	`RULES.md`
`/dgf-rules-check`	The change against the rules	nothing; a report
`/dgf-qa`	A manual test plan for the change	`qa/<plan>/test-plan.md`
`/dgf-docs`	The estate's inventory pages and overview	`docs/`
`/dgf-explore`	Cited research on a question	`RESEARCH.md`, after asking
`/dgf-grounded`	Answer only what it can cite	nothing
`/dgf-archive`	Archive finished plans	nothing; its script moves plans
`/dgf-security`	The application security scan	nothing; the scan writes reports

4.5 Parallel execution — Claude Code only

`claude --agent dgf-implement-coordinator` runs the **coordinator**, which asks `plan_layers.py` for the next layer of ready tasks, implements a one-task layer itself, and otherwise dispatches one **worker** per task, each in its own git worktree; it merges each back with `worktree_task.py`, runs the per-task check, ticks the plan, and ends with the verify gate's block (ADR 0033). A worker syncs the main tree's state, implements its one task and reports; it never ticks, commits, or edits a file its task does not list. A task joins a parallel batch only when its declared files are disjoint from every other's (four at once by default), and the merge copies back only declared files, refusing the whole task before anything moves if a worker wrote another file or the main tree changed underneath. Neither agent commits. Git is required. Elsewhere, `/dgf-implement` runs one task at a time.

5. The validators and the gate

5.1 Two schema families, resolved first

DGF has two configuration families: modern **JSON** (69 generated component schemas, vendored) and legacy **XML** (nine XSDs, vendored). `validate_config.py` resolves each file's family *before* parsing — an XML declaration or a known root is XML, a JSON object is JSON, anything else is `FAMILY_UNRESOLVED`, exit `3` — and **never falls back to JSON**, which would make every legacy file look like malformed JSON (ADR 0011).

JSON is validated the way the runtime reads it (ADR 0031): inherited properties merged, names and enums matched in any case, nested components checked against the class their own `type` selects, and an unknown property — which the runtime ignores — a warning. **Legacy XML:** `process.xml` is checked against `process.xsd` with a runtime-divergence allowance — a failure on a construct the runtime binds is a warning, any other failure blocks (ADR 0014 §1). Every other legacy grammar lags the runtime, so its failures are `XSD_LAGS_RUNTIME` warnings (ADR 0016); `profile.xsd` does not compile and is not checked. Malformed XML always blocks.

5.2 Runtime parity

A JSON file that validates is not proof the runtime reads JSON for it. A shipped table of 67 rows records, per component, whether the runtime reads its JSON fully (✓), partly (◐) or not at all (✗) — `Workflow` and `ProcessFlow` have JSON schemas the runtime ignores. Valid JSON for a ✗ or ◐ component is a warning (`PARITY_NOT_RUNTIME`, `PARITY_PARTIAL`) naming what is still read from XML (ADR 0011). The same table routes a *new* file (ADR 0010 §1): `route_means.py` answers JSON or XML. The five legacy types — form, workflow, process, settings, view — are always XML; a ✓ component is JSON; a ◐ one JSON with a warning; a component with no row has no route, never a guess. Existing files keep their family.

5.3 Components, processes and the data model

`resolve_components.py` checks that every component type, in JSON and in forms, is legal and that every referenced component file exists, by exact name. `validate_process.py` checks processes on files alone, with no database or engine (ADR 0014 §2):

Check	Severity
Dead transition — every transition and timeout target is a declared state or `End`	blocks
Unreachable state	warns — the engine can start an instance at any state
`WORKFLOW:` references in process, state, transition and MultiTask actions resolve	blocks
`validationFlow` resolves	blocks
A workflow's `CHANGE_STATE` names an existing process and its declared states	blocks

References resolve by the engine's own rules: one path per form, **exact case**, no fallback. A reference that resolves only on a case-insensitive filesystem is `CASE_ONLY_MATCH`, a warning. A reference in a `webasm` process that resolves in some applications only is `WORKFLOW_APP_DEPENDENT`, a warning naming both sets, because nothing in the root says which applications run that process.

`validate_model.py` checks the references an entity's `settings.xml` and a form make, resolved each loader's way, that the entity loads, and that every bound form cell names a field. **It blocks only where the loader throws** (ADR 0027 §3): a missing view, grid or form the runtime generates on demand is a warning.

`audit_root.py` builds a reference graph per application and reports the root's health, unresolved references no validator checks, the role inventory, or a file's **blast radius** — the processes that reach it in each application. It is a report, never a gate.

5.4 Change-relative gates

Real estates already carry findings — DGF's own samples raise hundreds — so a gate blocking on all of them would fail every branch. It blocks only on what the branch introduced (ADR 0018). `check_change.py` finds the merge-base, writes the base tree into a temporary folder blob by blob, and runs the same validators over the same scope at both trees. A finding at both is `INFO PRE_EXISTING`, one only at the base `INFO FIXED`; any other keeps its code and severity. Findings are compared by code, file and message — not line — and as a multiset, so a second copy is new. Without git or a merge-base the baseline is `NOT RUN` and every finding counts as new: stricter, never looser. The same run checks scope: each changed file must lie in a declared workspace, be planned, be of a permitted class, and follow its route. `/dgf-implement`'s per-task pre-check prints `NOT RUN: validators` and can never read as a pass. On a copy of DGF's samples, one added dead transition gave exactly one new error among 2552 pre-existing findings, in about five seconds for both runs (project documentation).

5.5 The gate block

`/dgf-verify` relays `verify_gate.py`, which runs the plan and change checks, audits the tasks, and ends with one `dgf-gate-result` block built by `scripts/lib/gate_result.py` (ADR 0022). Its rules:

- **Status is computed from two lists**: `fail` with any blocker, else `warn` with any warning, else `pass`; `blocking` is `true` exactly on `fail`. The builder takes no status and refuses a contradictory block.
- **It never claims a check it did not run.** Each required check — plan header, tasks, files, routes, Commit Plan; change scope, means, planned files; baseline; the whole-root validator run; frontend tests when a task is `kind: code` — is in `checks_run` or a `not-run-<check>` warning, never both.
- **Absent means not computed; `[]` means none.** The affected lists are the change's own footprint, not its blast radius.
- **`suggested_next`** names the command for the first thing blocking: `/dgf-plan`, `/dgf-implement`, `/dgf-fix` for a new finding once tasks are done and scope is clean, or `/dgf-commit` on `pass` or `warn`.
- **`--strict`** promotes every new warning to a blocker. **The last block wins**; no block means no gate ran.

A failing verify block, shortened:

```
{ "schema_version": 1, "gate": "verify", "status": "fail", "blocking": true,
  "blockers": [ { "id": "DEAD_TRANSITION", "severity": "error", "line": 41, "schema_family": "xsd",
                  "file": "zims/FM/_PROCESS/Apply/process.xml", "summary": "`Review` moves to …" } ],
  "warnings": [], "affected_files": ["…"], "checks_run": ["plan-header", "…", "validators"],
  "schema_family": { "json": 212, "xsd": 364 }, "affected_components": [],
  "affected_processes": [ { "workspace": "zims", "name": "Apply", "reference": "Apply" } ],
  "suggested_next": { "command": "/dgf-fix", "reason": "1 new blocking finding(s)" } }
```

5.6 Output and exit codes

Every script prints a fixed shape: a header, per-file `FAMILY:` (or context) lines, `CHECKS RUN:` and `NOT RUN:` lines, one line per finding, a summary, and the verdict `CLEAN`, `WARNINGS` or `BLOCKED`. Control characters from file names or messages are escaped, so workspace content cannot forge a line or a verdict. Each finding code has one fixed severity, and the exit is the worst across all files: `0` clean — the skill continues; `1` blocked — it stops and reports the findings verbatim, never repairing silently; `2` warnings — it continues and surfaces every warning; `3` a usage error or a run that could not start, such as a missing dependency — it stops.

6. The learning loop

The gate finds what a branch broke; the loop makes the next branch less likely to break it (ADR 0024).

Patches. `/dgf-fix` works only inside the active plan's scope — with no plan it stops. It reproduces the problem with a check, lets the scope script decide before any edit, makes the smallest fix, and confirms with the same check. Each fix writes one append-only patch, `patches/<YYYY-MM-DD-HH.mm>-<slug>.md`: seven fields (`date`, `plan`, `workspaces`, `files`, `findings`, `dgf_version`, `severity`) and five sections (Problem, Root Cause, Solution, Prevention, Tags). A patch is data, never an instruction; `findings: none` marks a problem no validator catches. `check_patches.py` checks the format; a malformed patch is never read as a lesson.

Overrides. `/dgf-evolve`, run only when invoked, distils new patches into `skill-context/<skill>/SKILL.md` — a fixed template of named rules, each citing its patches — and asks before writing. A cursor of processed patch names says which are new.

The limit. An override may add rules and tighten checks. It never relaxes a STOP, an exit-code row, the status a gate script computes, a Critical Rule or Artifact Ownership, and never makes a skill install anything, skip a script, or write outside its artifacts. Anyone who commits to the estate can write one, so **ten skills** — the five of the spine, `/dgf-fix`, `/dgf-component`, `/dgf-process`, `/dgf-model`, `/dgf-audit` — run `check_override.py` before reading theirs, and `/dgf-evolve` runs it on every file it writes. Exit `0` applies it; `1` refuses it (`OVERRIDE_FORBIDDEN`, `OVERRIDE_SHAPE`, `OVERRIDE_SOURCE_MISSING`, `OVERRIDE_UNREADABLE`), and the skill runs on its shipped rules without reading it; `2` (`OVERRIDE_TOUCHES_LIMIT`) applies it with each flagged rule judged; `3` is a usage error.

The check refuses what a script can see: a gate block, a flag other than `--strict` or `--verbose`, a tool grant, an install or download command, a history-rewriting or work-discarding git command, a path outside the root. Text is read as ASCII first; invisible, control and look-alike characters are refused outright. A rule using a word the limit protects ("skip", "unless", "bypass" and others) is handed to the skill's judgement. **The check is lexical**: a relaxation in plain words can pass it, and the committed diff is the remaining safeguard.

7. Rules, estate documentation, QA and security

The rules file. `/dgf-rules` writes `.dgf-factory/RULES.md`, each rule `## R<n> — <title>` with an `enforcement` of `script`, `gate` or `prompt-only`, a `check` for the first two, and a `source`. Six defaults ship (ADR 0028, ADR 0032 §4):

Rule	Enforcement
R1 — XML-always for the five legacy artifact types	script: the plan's route check
R2 — Declarative behaviour only	gate: code files must be a planned `kind: code` task
R3 — Base-workspace edits need explicit justification	gate: a plan touching `webasm` gives a reason
R4 — DGF-seam scope discipline; R6 — Test stack	prompt-only
R7 — Workspace layout	script: folder-name case against the framework's names

Estate rules start at R8 and may only add a constraint: prompt-only, or citing an existing check — never a new script. `check_rules.py` holds the file to the override limit, and a forbidden construct refuses the whole file, so the shipped defaults apply. `/dgf-plan`, `/dgf-implement`, `/dgf-review` and `/dgf-rules-check` run it first.

Estate documentation. `/dgf-docs` runs `estate_docs.py`, which renders an index and a page per workspace — processes, states and transitions, workflows, entities and relations, components, forms, role names — with the validators' own readers, deterministically (sorted, no timestamp, no absolute path). A page without the generated marker is never overwritten; `--check` reports stale pages with exit `2`, for CI.

QA and research. `/dgf-qa` writes a scenario per process the change reaches, in each application, and names automated tests only for `kind: code` tasks. `/dgf-explore` saves cited research that `/dgf-plan` reads as data, never instructions.

The security scan (ADR 0034). `/dgf-security` runs the shipped `security-scan.sh` — the only shell script a skill runs — over the application's code and host, not its DGF configuration. Eight steps: runtime and framework versions (.NET, Node.js, Angular, React, Next.js still supported and patched); NuGet and npm dependencies against advisories (`dotnet`'s and npm's own audits, and Trivy); secrets in files and git history, redacted in the report (Gitleaks); Dockerfile, Kubernetes, Helm, Terraform and `web.config` settings (Trivy); source code (Semgrep); container images, with `--image` or `--build` (Trivy); an OWASP ZAP *baseline* of a running test site, with `--url`, which does not attack it; and a CycloneDX SBOM. Each scanner is used when installed, else its version-pinned Docker image; with neither the step is a `SKIP` row and the run exits `2` — a skipped check is never a pass. It needs the network for advisories, release data and images. Reports go under `.dgf-factory/security/`, and a folder the scan creates gets a `.gitignore` of `*`. Its header states macOS, Linux and WSL; there is no Windows-native version.

8. Prerequisites

Requirement	Needed for	Notes
Node.js ≥ 18.3	installing and updating	not needed by a teammate who clones a set-up project
GitHub token with `read:packages`	installing from GitHub Packages	once per machine
Python 3.9+	every validator and the gate	setup builds `.venv`; ADR 0031 notes 3.9 is end-of-life upstream
Network	`pip` at setup, the hosted `dgf-mcp` docs server, the security scan	`dgf-mcp` needs no credentials
An AI harness	running the skills	one of the 16 in §2.3, started at the project root
Git	branches, the change-relative gate, parallel runs	without it, every finding counts as new
Claude Code	parallel execution; full interactive behaviour	see §9
Docker, or Trivy, Gitleaks, Semgrep, ZAP	the security scan only	optional; without them steps are skipped
`dotnet` and Docker	building and running a scaffolded application	not needed by the pipeline

No DGF repository checkout is needed. No operating-system support matrix is declared: the CLI handles POSIX and Windows virtual-environment layouts, the release workflow runs the tests on Ubuntu, and the security scan states macOS, Linux and WSL.

9. Documented limits

Static checks only; no runtime behaviour (ADR 0014 §5). A process cannot be executed headlessly, so the gate's claim is bounded: the files match the grammar the runtime reads and are free of the defects in `checks_run`. A process can pass every check and still fail in the browser; browser tests stay outside the gate. Unreachable states only warn, so some real dead states will ship.

Roles are an inventory, not a check (ADR 0027 §6; knowledge base `permissions.md` §1). Roles are ASP.NET Identity database rows; nothing in a workspaces root declares one. The audit lists every role name the root uses, exactly as the runtime tests it, and checks none, so a misspelt role is never reported.

Warning-only references. A `webasm` reference that resolves in some applications only is `WORKFLOW_APP_DEPENDENT`, a warning; DGF's samples produce at least 29 (ADR 0014). A workflow's form references and the workflow edges `sub-workflow`, `invoke-table`, `record-table`, `island-table` and `datasource-step` are reported by the audit only, never by a gate; promoting them is a roadmap follow-up. Legacy grammars other than `process.xsd` only warn (ADR 0016), `profile.xsd` is not checked, and unknown JSON properties warn (ADR 0031). ADRs 0010, 0014, 0016 and 0031 each note that such common warnings get ignored.

Configuration plus a little JavaScript or CSS — no C# (ADR 0010). The pipeline composes JSON first, legacy XML where no JSON route exists, and JavaScript or CSS only for a task marked `kind: code` with a reason. C#, TypeScript, Angular, SQL, HTML and plugin assemblies are excluded: a plan names them under Scope and no task writes them, so such a feature is split across tools. Only a form's `_form.js` may be new; other `.js` and `.css` files are edit-only, because nothing in the root makes the UI load a new one. "A little" has no number and is judged at review. Code tasks get a weaker gate: frontend tests are required but never run by the gate, so a plan with a code task reports `not-run-frontend-tests`. `/dgf-model` names database work and writes no SQL. The one exception is `/dgf-scaffold`, a template-only generator that writes C#, SQL, compose and MSBuild into an empty folder; ADR 0027 §7 records that this rests on reading ADR 0010 as the pipeline's scope, a reading a reviewer may reject.

The scaffold's output has not been run against a live DGF (roadmap; ADR 0027 §7.12). It builds and checks clean; its solution files pass no validator, only the post-generation check and, where `dotnet` exists, a build. It does not run until its `${VAR}` placeholders are filled and four inputs DGF pins no artifact for are supplied: the base workspace, the image publishing the framework's database baseline, the DGF UI image, and a NuGet feed serving `DGF.API`. It builds no domain beyond its three module patterns.

Blast radius is incomplete (ADR 0027 §8). A static walk cannot see names assigned at run time, processes chosen from the database, or entry points outside processes (profile trees, sitemaps, form and view `WORKFLOW:` strings, JSON `workflow` components). Every reach report ends with a `LIMIT:` line, and an unreached shared workflow is a candidate, never proof of dead code.

Knowledge stamped to one DGF version (ADR 0013, ADR 0019). Every knowledge file is stamped `1.1.11`, which ADR 0013 notes understates the DGF commit actually read. `/dgf` records the estate's declared version and warns when it is newer, but nothing checks that value against a deployment, and

the version gate comparing it with a fact's range is not built. There is no upgrade or migration skill; both are guided routes (ADR 0034).

Other harnesses run degraded skills (ADR 0029 §8; roadmap follow-ups). Script output and `suggested_next` name commands in Claude Code's `/dgf-…` form, so a Codex user reads `/dgf-fix` where they type `$dgf-fix`. Skills ask questions through Claude Code's `AskUserQuestion` tool and `/dgf-plan` delegates to an `Explore` agent; other harnesses read both as plain instructions. Skills assume the harness started at the project root. Three harnesses need the MCP entry added by hand.

Parallel execution is Claude Code only, and its dispatch is untested here (ADR 0033). The scheduling and merge scripts are tested; agent behaviour is not testable in this repository. A plan that under-declares files runs serially or is refused, and each worker pays a full checkout.

Change-relative comparison (ADR 0018). Without git every finding blocks. A pre-existing finding made worse without changing its message reads as pre-existing.

Judgement stays judgement (ADR 0024, ADR 0032). The override and rules checks are lexical, and a review's or a rules check's verdict is labelled as the model's judgement, never a gate.

10. Security posture

Where it writes. `setup` writes only inside the project: every target must be project-relative, and one a symlink would take outside is refused; it refuses the filesystem root and the home directory. Outside the project, only what `npm -g` installs is written. Every file is in the receipt with two hashes; an edited file is never overwritten or removed; files the tool does not own are parsed and merged, never replaced; the project's `.gitignore` is never edited.

What it depends on. The CLI has no runtime dependencies. The validators' packages are hash-pinned and installed with `pip --require-hashes` into the project's own environment, and pip never runs outside it. XML is parsed with entity resolution, network access and DTD loading off (ADR 0031). Apart from the `pip` step, neither the CLI nor the Python scripts open a network connection; the overlap check reads other branches' plans from local refs and never fetches.

Secrets appear only as `${VAR}` placeholders (ADR 0029 §6), and `dgf-mcp` needs no credentials. The scaffold's post-generation check blocks a literal secret and an undeclared variable. The security scan redacts secrets in its report, and its reports are kept out of commits.

What each skill may run. The read-only skills — `/dgf-verify`, `/dgf-audit`, `/dgf-doctor`, `/dgf-review`, `/dgf-rules-check`, `/dgf-grounded` — pre-approve no write tool. **No skill pre-approves a git command**: a rule broad enough for `git -C "<root>" …` would also pre-approve `git -c alias.x="!…"`, which runs any command, so every git command is shown to the user and approved in the harness's own prompt (ADR 0035). Each script a skill may run without a prompt is named one by one, from the installed runtime, so no other file can run pre-approved. `/dgf-commit` commits and pushes only after the user confirms. Anything not pre-approved still asks.

Estate content is data. Overrides and the rules file are checked by a script before any skill reads them, and a refused file is never read. Patches and research are read as data, never instructions. Another branch's plan is named, never quoted, since anyone who can push a branch writes it. Script output escapes control characters, so file content cannot forge a finding or a verdict. A parallel worker's files are merged only if its task declares them.

Appendix A — Scripts and exit codes

Shipped in `.dgf-factory/runtime/scripts/`, except the three marked *, which live in their skill's own `scripts/` folder. Every script exits `0` clean and `3` on a usage error or when it could not run (for example, missing dependencies); the columns give what `1` and `2` mean for each.

Script	Decides	`1` blocked	`2` warnings
`validate_config.py`	family, schema, parity	invalid, malformed, unknown component	parity, unknown property, XSD lag
`resolve_components.py`	component types and files	unknown type, missing file	case-only, app-dependent
`validate_process.py`	processes, workflow targets	bad structure, dead transition, unresolved	unreachable, app-dependent
`validate_model.py`	entities and forms	unresolved, unloadable, unbound cell	generated on demand, app-dependent
`route_means.py`	JSON or XML for a new file	—	JSON, partly read from XML
`locate_plan.py`	the root and the active plan	missing or ambiguous	fell back to another plan
`inventory_root.py`	each workspace, counted	—	no base workspace, no dependencies
`check_plan.py`	a plan's format, scope, routes	a plan defect	overlap, partial parity
`check_change.py`	scope; new findings	out of scope, new blocking finding	unplanned file, new warning
`verify_gate.py`	the verify gate and its block	`fail`	`warn`
`check_patches.py`	patch format; which are new	malformed patch	cursor unreadable
`check_override.py`	may a skill read its override	refused	touches the limit
`check_rules.py`	may a skill read the rules	refused, layout case	touches the limit, default edited

Script	Decides	1 blocked	2 warnings
`audit_root.py`	health, graph, roles, reach	a validator error	unresolved reference
`show_change.py`	the branch's diff	—	—
`estate_docs.py`	the estate's inventory pages	page without the marker	stale page
`archive_plans.py`	archives finished plans	incomplete plan, destination exists	plan skipped, nothing to do
`plan_layers.py`	the next layer of tasks	dependency cycle or missing	—
`worktree_task.py`	a worker's sync and merge	undeclared file, main tree changed	—
`security-scan.sh`	application security	a blocking finding	a warning, a skipped step
`doctor.py` *	the install	missing file or skill, DGF path shipped	edited file, version drift
`design.py` *	the scaffold's design	a conflict	values still open
`generate.py` *	renders, then checks, an app	target exists, check failed	—

Appendix B — Glossary

Term	Meaning
Workspaces root	The folder holding a DGF estate's workspaces; the unit the gate checks
Workspace	A folder with an `FM/` configuration tree: `webasm` is the base, the others applications
Runtime parity	Whether the DGF runtime actually reads a component's JSON: ✓, ◐ or ✗
`kind: code`	A plan task that writes a little JavaScript or CSS, with a stated reason
`PRE_EXISTING`	A finding present at the merge-base; it never affects the exit
Blast radius	The processes that reach a file, per application; a report, never complete
Patch	A record of one fix or gotcha, written by `/dgf-fix`
Override	Project rules for one skill, written by `/dgf-evolve`, that may only tighten it
Receipt	`.dgf-factory/install.json`: every file setup wrote, with two hashes
Knowledge stamp	The DGF version and read date every knowledge file carries